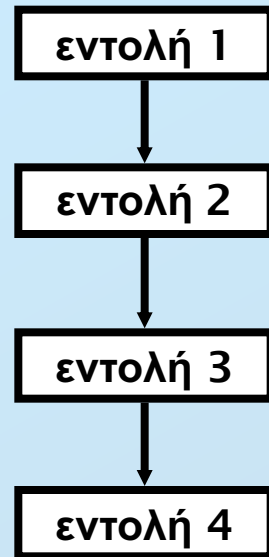
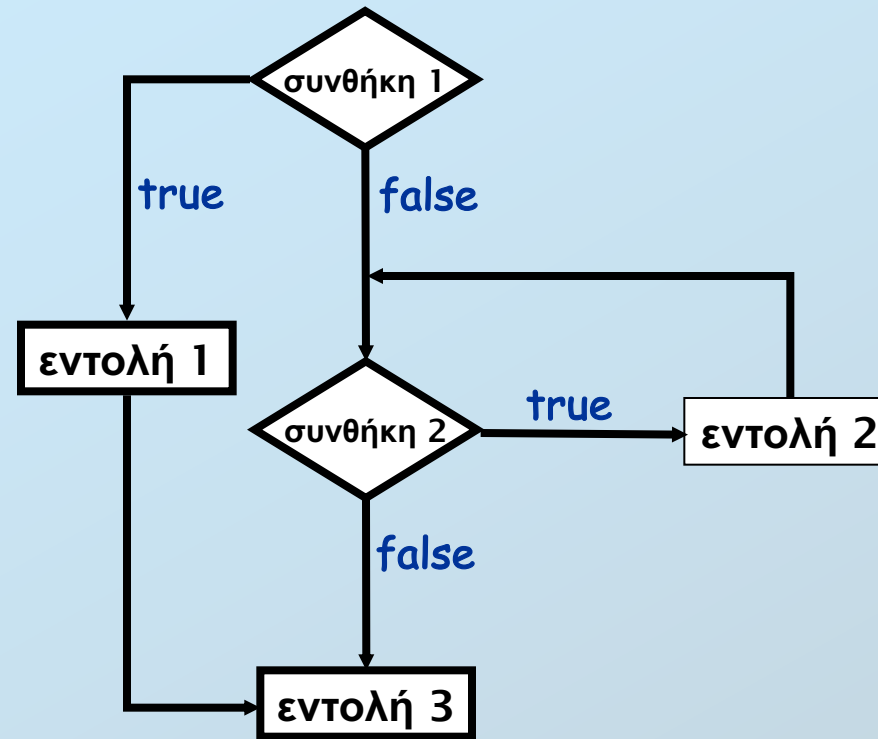


# **Αλγοριθμικές Δομές Επιλογής και Επανάληψης**

**Συνθήκες, Λογικές Εκφράσεις  
και Βρόγχοι**



**ακολουθιακή ροή  
ελέγχου**



**ροή ελέγχου με συνθήκες και  
βρόγχους**

## Συνθήκη – Condition

- Μία έκφραση η οποία είναι είτε **αληθής** (true) είτε **ψευδής** (false)
- Δηλαδή μία **λογική έκφραση**

**Προσοχή:** Δοκιμή για ισότητα ανάμεσα σε πραγματικούς αριθμούς

```
1.29999 == 1.2999 ?
```

```
⇒ false
```

```
Math.abs(1.29999 - 1.2999) < 0.0001
```

```
⇒ true
```

## Αποτίμηση Λογικών Εκφράσεων

Η αποτίμηση αρχίζει από τα αριστερά και προχωρεί μέχρι το σημείο που χρειάζεται να προχωρήσει (**short-circuit evaluation**), π.χ.

`false && E  $\Rightarrow$  false`

η αποτίμηση της έκφρασης E δεν χρειάζεται

`true || E  $\Rightarrow$  true`

η αποτίμηση της έκφρασης E δεν χρειάζεται

## Σειρά Συνθηκών σε μία Λογική Έκφραση

Ο κανόνας αποτίμησης είναι κυρίως για λόγους **αποδοτικότητας**

Έχει όμως και μεγάλη χρησιμότητα αναφορικά με τη **σειρά των συνθηκών** σε μία λογική έκφραση, π.χ.

$$(y \% x) > 2 \ \&\& \ x \neq 0$$

Εάν το  $x$  ισούται με 0, η αποτίμηση της πιο πάνω έκφρασης οδηγεί σε λάθος. Όμως η ακόλουθη διατύπωση

$$x \neq 0 \ \&\& \ (y \% x) > 2$$

δεν έχει πρόβλημα λόγω του κανόνα αποτίμησης:

Εάν το  $x$  ισούται με 0 η συνθήκη  $(y \% x) > 2$  δεν αποτιμάται

Εάν το  $x$  δεν ισούται με 0, η αποτίμηση της έκφρασης  $(y \% x) > 2$  δεν δημιουργεί πρόβλημα

## Σειρά Συνθηκών σε μία Λογική Έκφραση

Γενικά σε μία σύζευξη,  $E_1 \&\& E_2$ , η συνθήκη  $E_1$  πρέπει να αποτελεί τον αριστερό τελεστέο, εάν στην περίπτωση που αποτιμάται σε `false`, τυχόν αποτίμηση της  $E_2$  θα οδηγήσει σε πρόβλημα. Με αυτή την σειρά αποτρέπεται η αποτίμηση της  $E_2$ .

Παρομοίως σε μία διάζευξη,  $E_1 \parallel E_2$ , η συνθήκη  $E_1$  πρέπει να αποτελεί τον αριστερό τελεστέο, εάν στην περίπτωση που αποτιμάται σε `true`, τυχόν αποτίμηση της  $E_2$  θα οδηγήσει σε πρόβλημα. Με αυτή τη σειρά αποτρέπεται η αποτίμηση της  $E_2$ .

## Εκμετάλλευση Κανόνα Αποτίμησης Συνθηκών για σκοπούς Αποδοτικότητας

Ακόμη και σε περιπτώσεις όπου η σειρά των συνθηκών σε μία σύζευξη/διάζευξη θα μπορούσε να είναι οποιαδήποτε, ενδείκνυται να γίνεται εκμετάλλευση του κανόνα αποτίμησης προς μείωση του υπολογιστικού κόστους.

Π.χ. έστω η σύζευξη ( $E_1 \ \&\& \ E_2$ ) όπου το κόστος υπολογισμού (αποτίμησης) της  $E_1$  είναι πολύ μεγάλο, ενώ της  $E_2$  είναι σχετικά πολύ μικρό.

Έστω ότι η  $E_1$  αποτιμάται σε true και η  $E_2$  σε false. Η πιο πάνω σειρά των συνθηκών δεν αποφεύγει το μεγάλο κόστος αποτίμησης της  $E_1$ . Όμως η σειρά  $E_2 \ \&\& \ E_1$  το αποφεύγει.

Επομένως η ορθή πρακτική είναι να μπαίνει στα αριστερά η συνθήκη, της οποίας η αποτίμηση συνεπάγεται το **λιγότερο κόστος**

## Προτεραιότητες

κλήσεις μεθόδων

! + − (μοναδιαίοι τελεστές)

\* / %

+ −

< <= >= >

== !=

&&

||

=

*υψηλότερη προτεραιότητα*



*χαμηλότερη προτεραιότητα*

## Σύγκριση Χαρακτήρων

Οι σχεσιακοί τελεστές μπορούν να εφαρμοστούν σε χαρακτήρες

**έκφραση**

**τιμή**

`'9' >= '0'`

`true`

`'a' < 'e'`

`true`

`'B' <= 'A'`

`false`

`'Z' == 'z'`

`false`

`'a' <= 'A'`

`false`

## Ανάθεση Λογικών Τιμών

```
boolean even, odd;  
even = (n % 2 == 0);  
ή even = n % 2 == 0;  
odd = !even;
```

```
boolean is_lower, is_upper, is_letter, white_space;  
char ch = '!';  
is_lower = 'a' <= ch && ch <= 'z';  
is_upper = 'A' <= ch && ch <= 'Z';  
is_letter = is_lower || is_upper;  
white_space = ch == '\n' || ch == ' ' || ch == '\t' ;
```

## Συμπλήρωμα Συνθηκών – Complementing Conditions

Το συμπλήρωμα μίας συνθήκης είναι η άρνησή της, π.χ.

### Συνθήκη

```
true  
false  
item == SENT  
item > SENT  
(n % 2) == 0
```

### Συμπλήρωμα

```
false  
true  
!(item == SENT)   ή   item != SENT  
item <= SENT  
(n % 2) != 0
```

## Κανόνες DeMorgan

$$\neg(E_1 \ \&\& \ E_2) \equiv \neg E_1 \ \vee \ \neg E_2$$

$$\neg(E_1 \ \vee \ E_2) \equiv \neg E_1 \ \&\& \ \neg E_2$$

### Απόδειξη

| $E_1$ | $E_2$ | $\&\&$ | $\neg$ | $\neg E_1$ | $\neg E_2$ | $\vee$ |
|-------|-------|--------|--------|------------|------------|--------|
| T     | T     | T      | F      | F          | F          | F      |
| T     | F     | F      | T      | F          | T          | T      |
| F     | T     | F      | T      | T          | F          | T      |
| F     | F     | F      | T      | T          | T          | T      |

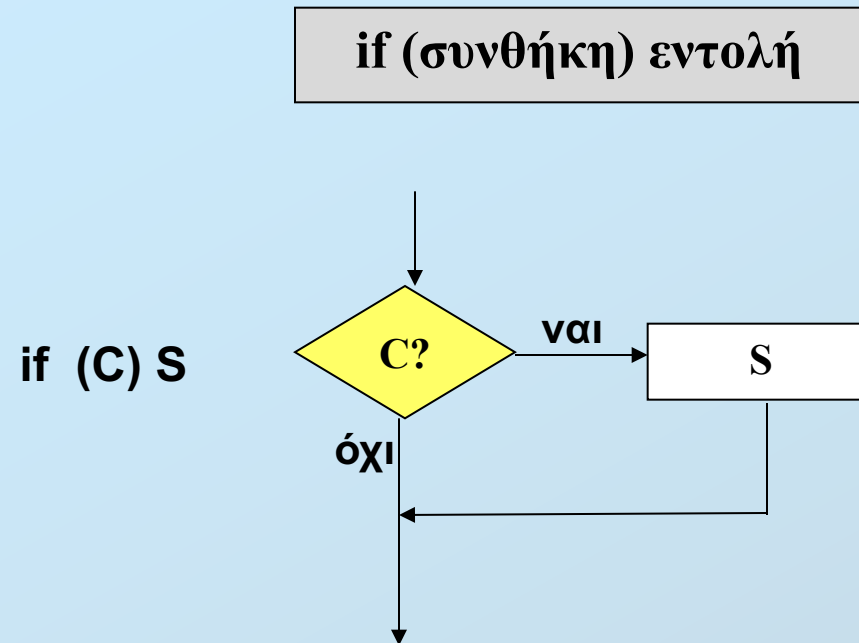
| $E_1$ | $E_2$ | $\vee$ | $\neg$ | $\neg E_1$ | $\neg E_2$ | $\&\&$ |
|-------|-------|--------|--------|------------|------------|--------|
| T     | T     | T      | F      | F          | F          | F      |
| T     | F     | T      | F      | F          | T          | F      |
| F     | T     | T      | F      | T          | F          | F      |
| F     | F     | F      | T      | T          | T          | T      |

## Παραδείγματα

```
!(age > 25 && (status == 'S' || status == 'D'))  
≡ !(age > 25) || !(status == 'S' || status == 'D')  
≡ age <= 25 || (status != 'S' && status != 'D')
```

```
!(year % 4 == 0 && (year % 100 != 0 || year % 400 == 0))  
≡ !(year % 4 == 0) || !(year % 100 != 0 || year % 400 == 0)  
≡ year % 4 != 0 || !(year % 100 != 0) && !(year % 400 == 0)  
≡ year % 4 != 0 || year % 100 == 0 && year % 400 != 0
```

## Εντολή if

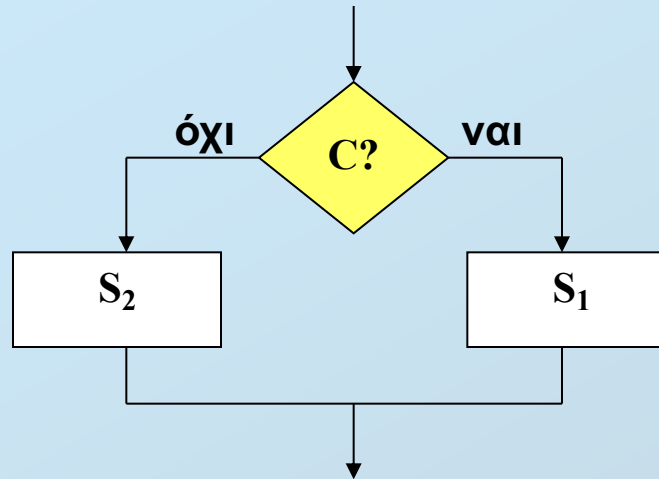


```
if (i == 1) N = N/10;
```

```
if (i == 1) { D = N % 10; N = N / 10; }
```

if (συνθήκη) εντολή<sub>1</sub>  
else εντολή<sub>2</sub>

if (C) S<sub>1</sub>  
else S<sub>2</sub>



```
if (x == 0) D = 0;  
else D = D / x;
```

```
if (x > 0.0) {y = Math.sqrt(x);  
             z = Math.sqrt(y);}  
else { y = 0.0;  z = 0.0; }
```

```
if (x == 0); /* do nothing */  
else y = 1/x;
```

≡ `if (x != 0) y = 1/x;`

```
if (x > 5) y = x * 2;  
else ; /* do nothing */
```

≡ `if (x > 5) y = x * 2;`

## Γενικά

`if (C); /* do nothing */`

`else S`

≡ `if (!C) S`

Χρησιμοποιείται εάν είναι πιο φυσικό να εκφραστεί η συνθήκη C παρά η άρνησή της

`if (C) S else; /* do nothing */`

≡ `if (C) S`

Δεν έχει νόημα να υπάρχει κενό `else`

## Εντολή switch

```
switch (έκφραση_ελέγχου) {  
    σύνολο_ετικετών_1  
        εντολές_1  
        break;  
    σύνολο_ετικετών_2  
        εντολές_2  
        break;  
    .  
    .  
    σύνολο_ετικετών_n  
        εντολές_n  
        break;  
    default:  
        εντολές_d  
}
```

Η έκφραση\_ελέγχου πρέπει να είναι βαθμωτού τύπου, π.χ. char ή int, αλλά όχι double

Μπορεί να είναι String αλλά όχι boolean

Η εκτέλεση τερματίζεται όταν φτάσουμε σε break

## Παραδείγματα

```
char choice = args[0].charAt(0);

switch(choice) {
    case 'B':
    case 'b': System.out.println("B or b");
               break;
    case 'W':
    case 'w': System.out.println("W or w");
               break;
    case 'K':
    case 'k': System.out.println("K or k");
               break;
    case 'R':
    case 'r': System.out.println("R or r");
               break;

    default: System.out.println("Wrong choice");
}
}}
```

Είσοδος: k  
Έξοδος: K or k

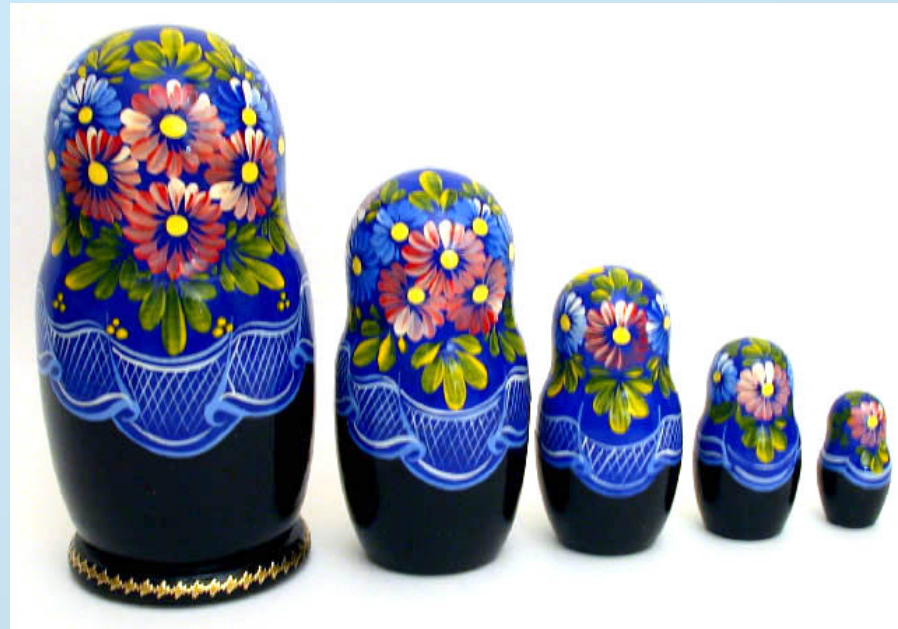
Είσοδος: W  
Έξοδος: W or w

Είσοδος: x  
Έξοδος : Wrong choice

```
switch (year % 4 == 0) {  
  
    case false: isLeapYear = false;  
                break;  
  
    case true:  switch (year % 100 == 0) {  
  
                case false: isLeapYear = true;  
                            break;  
  
                case true:  switch (year % 400 == 0) {  
  
                            case false: isLeapYear = false;  
                                    break;  
  
                            case true:  isLeapYear = true;  
                                    }  
                }  
    }  
}
```

Το πιο πάνω δεν επιτρέπεται

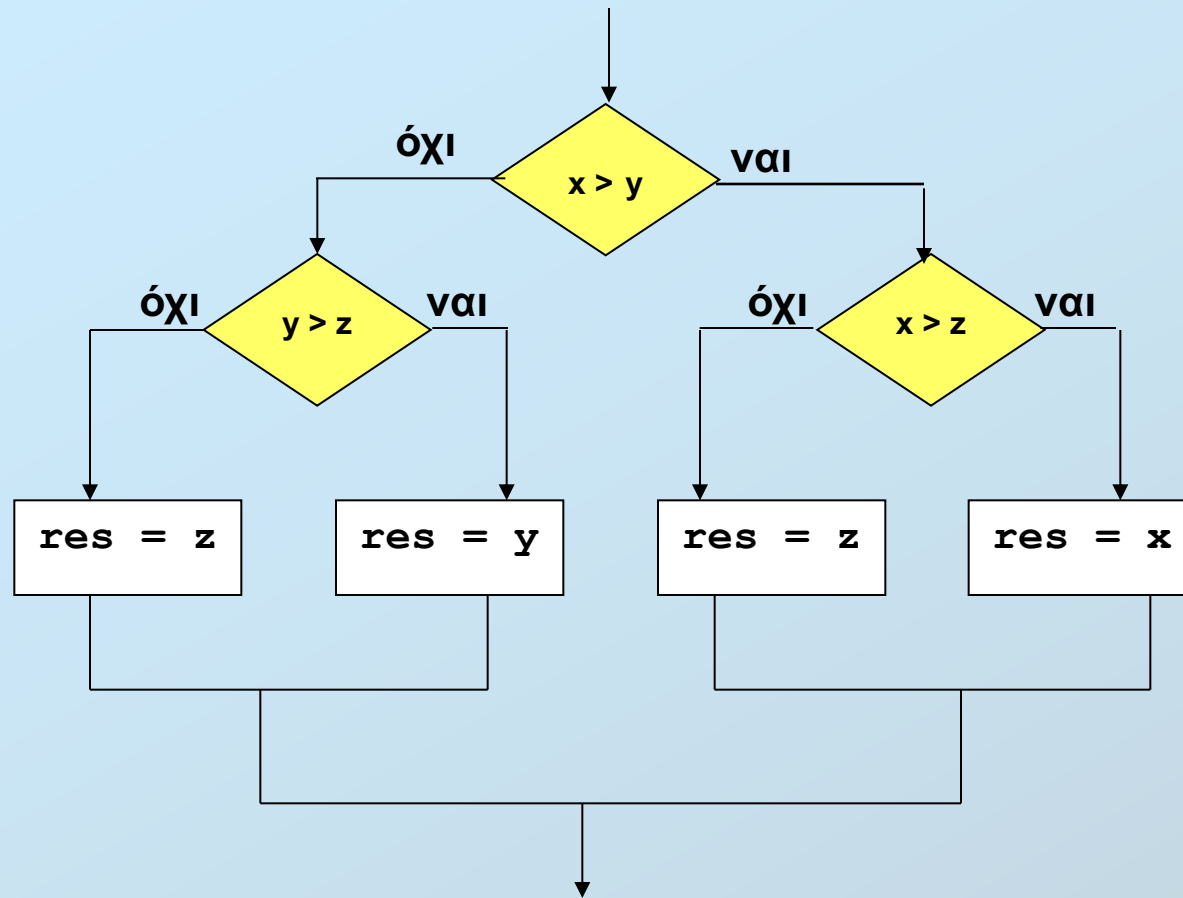
## Φώλιασμα Επιλογών - Nesting



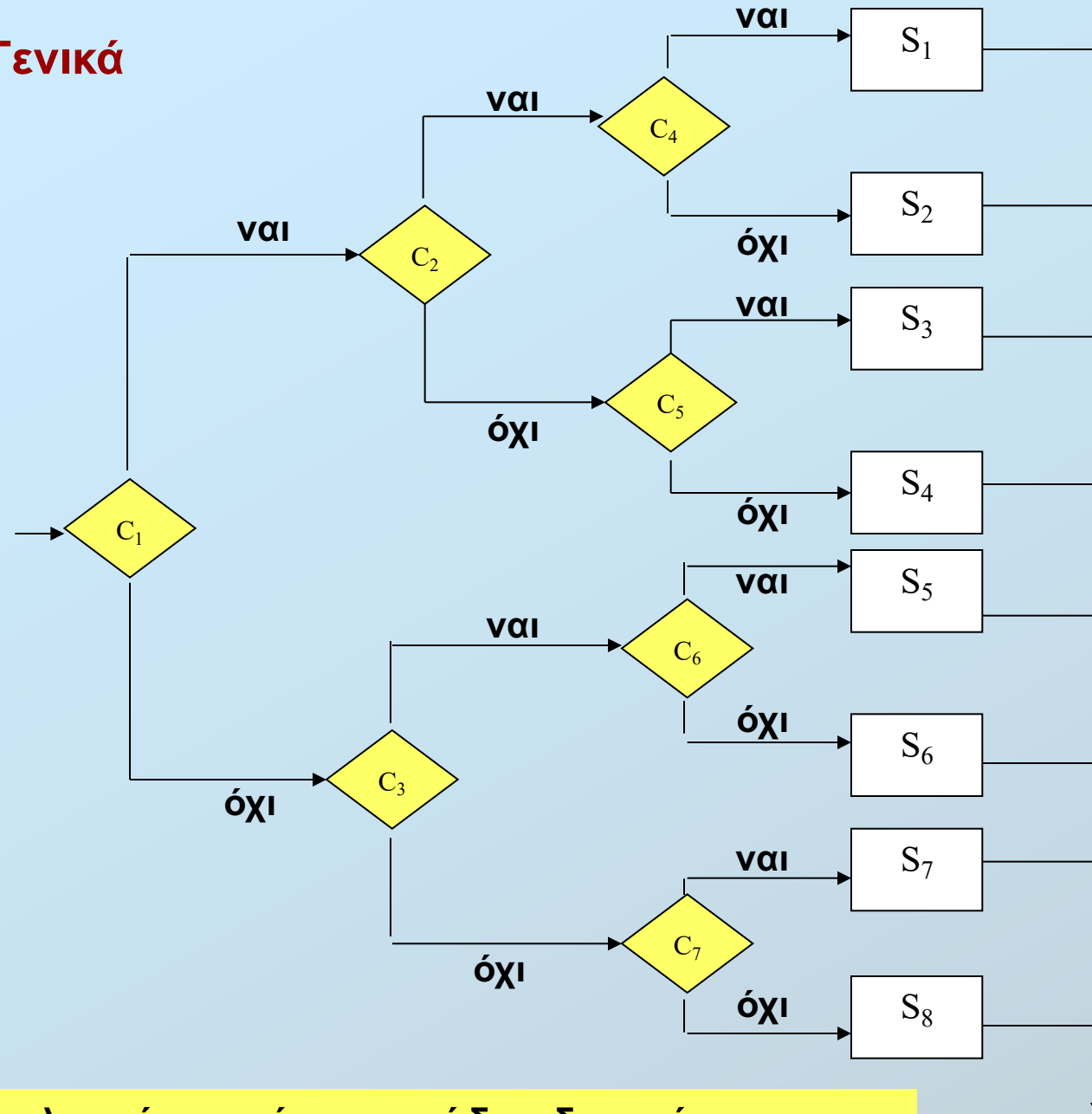
## Φώλιασμα Επιλογών

```
if (x > y)
    if (x > z)    res = x;
    else        res = z;
else if (y > z)   res = y;
else             res = z;
```

Τί σημαίνει το πιο πάνω φώλιασμα επιλογών;



## Γενικά



```

if (C1)
    if (C2)
        if (C4)    S1
        else      S2
    else
        if (C5)    S3
        else      S4
else
    if (C3)
        if (C6)    S5
        else      S6
    else
        if (C7)    S7
        else      S8

```

```

if      (C1 && C2 && C4)    S1
else if (C1 && C2 && !C4) S2
else if (C1 && !C2 && C5) S3
else if (C1 && !C2 && !C5) S4
else if (!C1 && C3 && C6) S5
else if (!C1 && C3 && !C6) S6
else if (!C1 && !C3 && C7) S7
else if (!C1 && !C3 && !C7) S8

```

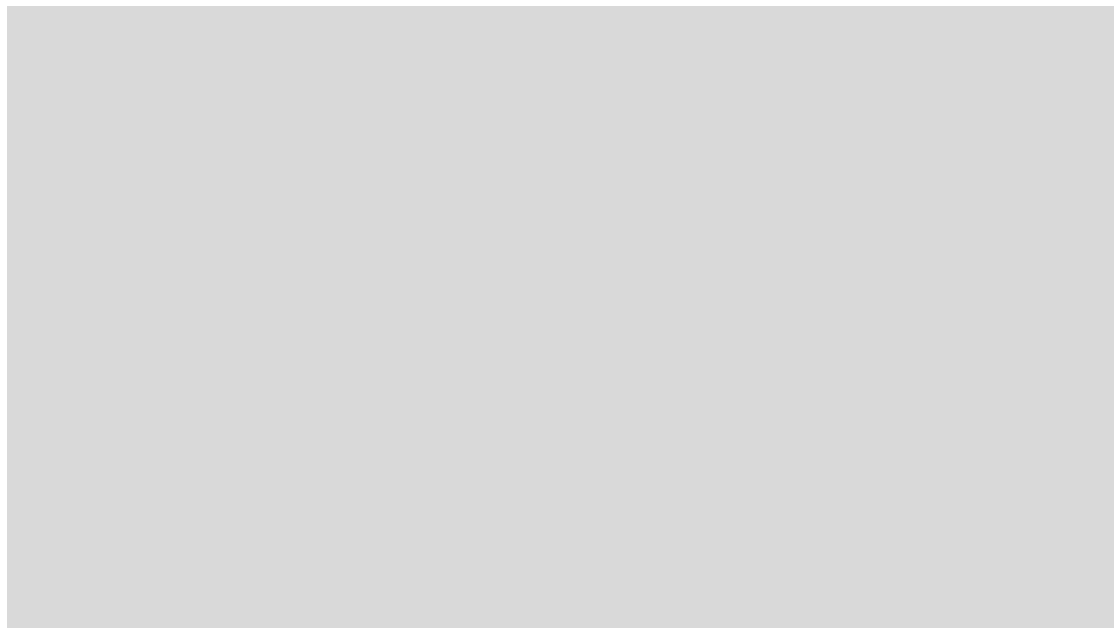
**Πιο διαφανής λογική**

**Όμως η κάθε συνθήκη αποτιμάται  
πολλαπλάσιο αριθμό φορές**

## Επανάφορά στο Παράδειγμα

Κατά πόσο δεδομένο έτος είναι δίσεκτο ή όχι

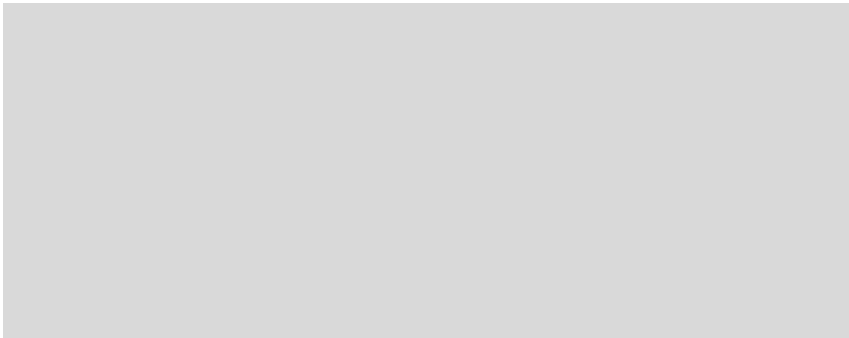
```
if (year % 4 == 0)
```



```
else isLeapYear = false;
```

## Επαναφορά στο Παράδειγμα

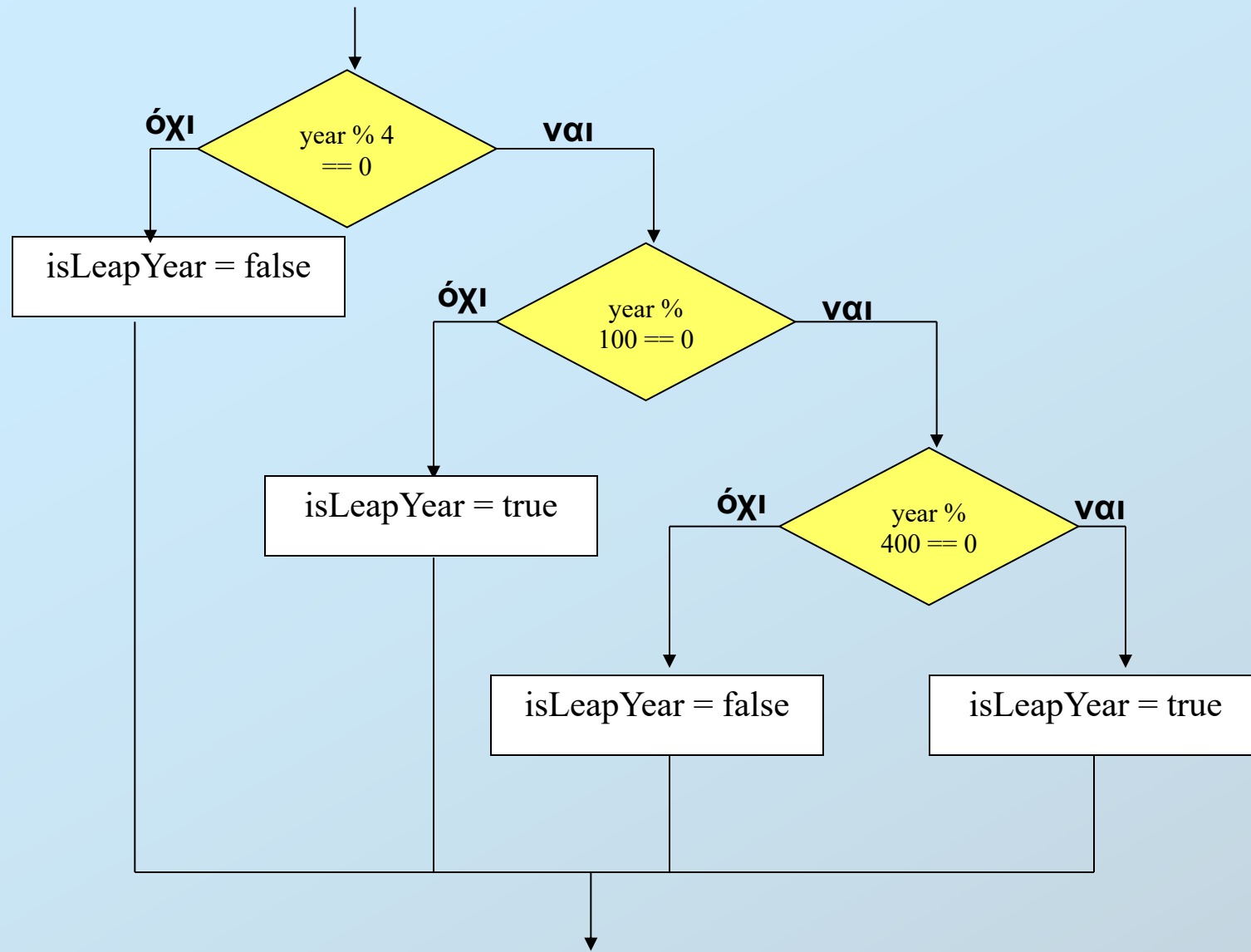
Κατά πόσο δεδομένο έτος είναι δίσεκτο ή όχι

```
if (year % 4 == 0)
    if (year % 100 == 0)
        
    else isLeapYear = true;
else isLeapYear = false;
```

## Επανάφορά στο Παράδειγμα

Κατά πόσο δεδομένο έτος είναι δίσεκτο ή όχι

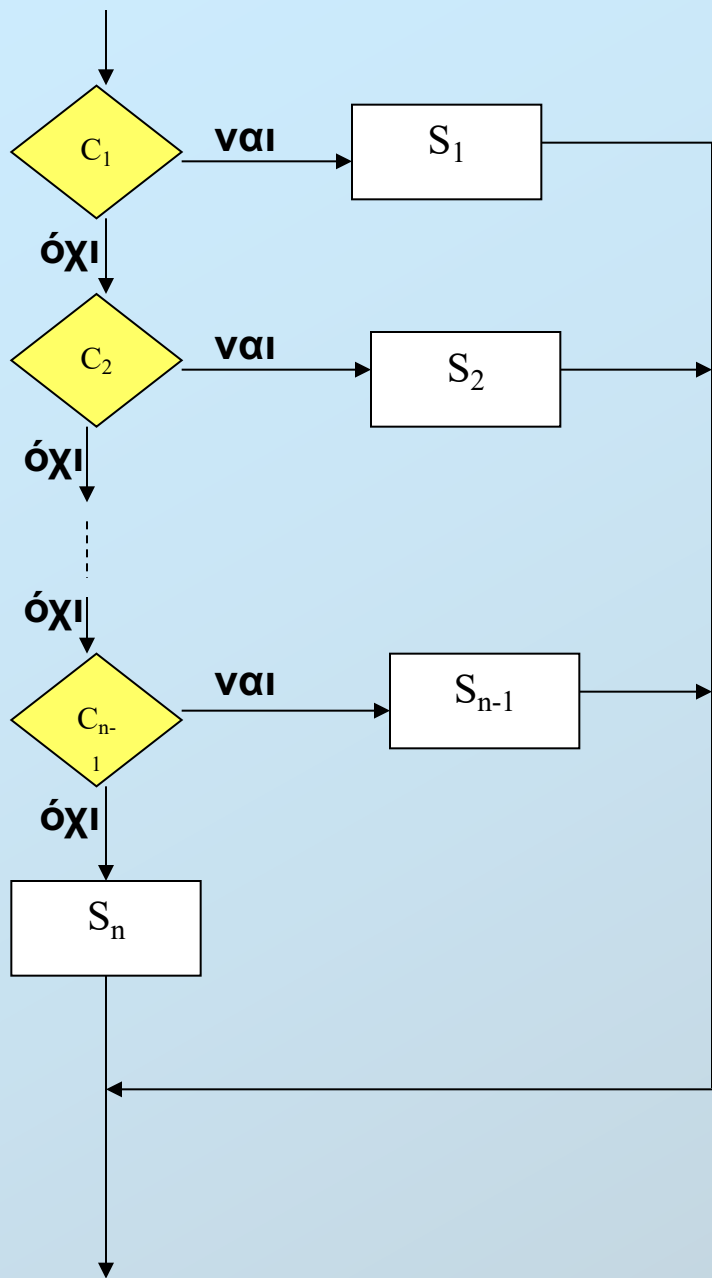
```
if (year % 4 == 0)
    if (year % 100 == 0)
        if (year % 400 == 0)
            isLeapYear = true;
        else isLeapYear = false;
    else isLeapYear = true;
else isLeapYear = false;
```



```
if      (year % 4 == 0 && year % 100 == 0 && year % 400 == 0)
    isLeapYear = true;
else if (year % 4 == 0 && year % 100 == 0 && year % 400 != 0)
    isLeapYear = false;
else if (year % 4 == 0 && year % 100 != 0)
    isLeapYear = true;
else if (year % 4 != 0)
    isLeapYear = false;
```

### **Διακλάδωση μόνο στην κατεύθυνση του όχι**

Ο πιο συνηθισμένος τρόπος φωλιάσματος επιλογών, απλά επειδή αποτελεί τον πιο φυσικό τρόπο έκφρασης, είναι όταν η διακλάδωση γίνεται μόνο στην κατεύθυνση του όχι. Έτσι υπάρχει μία αλυσίδα επιλογών.



if ( $C_1$ )  $S_1$   
 else if ( $C_2$ )  $S_2$   
 else if ( $C_3$ )  $S_3$   
 .....  
 else if ( $C_{n-1}$ )  $S_{n-1}$   
 else  $S_n$

## Παραδείγματα

```
if      (x > z  &&  x > y)      res = x;
else if (y > x  &&  y > z)      res = y;
else                                     res = z;
```

```
if (year % 4 == 0  &&  year % 100 != 0)
    isLeapYear = true;
else if (year % 4 == 0  &&  year % 100 == 0  &&  year % 400 == 0)
    isLeapYear = true;
else  isLeapYear = false;
```

```
if      (weighted_aver >= 8.5)
    System.out.println("Excellent");
else if (weighted_aver < 8.5  &&  weighted_aver >= 6.5)
    System.out.println("Very Good");
else if (weighted_aver < 6.5  &&  weighted_aver > 5.0)
    System.out.println("Good");
else  System.out.println("Failure");
```

## Παράδειγμα: Υπολογισμός φορολογικού συντελεστή βάσει εισοδήματος

| Εισόδημα          | Φορολογικός συντελεστής |
|-------------------|-------------------------|
| 0 - 47,449        | 22%                     |
| 47,450 - 114,649  | 25%                     |
| 114,650 - 174,699 | 28%                     |
| 174,700 - 311,949 | 33%                     |
| 311,950 -         | 35%                     |

5 αμοιβαία  
αποκλειόμενες  
επιλογές

```
double rate;  
if      (income < 47450) rate = 0.22;  
else if (income < 114650) rate = 0.25;  
else if (income < 174700) rate = 0.28;  
else if (income < 311950) rate = 0.33;  
else      rate = 0.35;
```

Η πιο κάτω διατύπωση είναι ορθή ή λάθος;

```
double rate = 0.35;  
if (income < 47450) rate = 0.22;  
if (income < 114650) rate = 0.25;  
if (income < 174700) rate = 0.28;  
if (income < 311950) rate = 0.33;
```

Η πιο κάτω διατύπωση είναι ορθή ή λάθος;

```
double rate = 0.35;  
if (income < 311950) rate = 0.33;  
if (income < 174700) rate = 0.28;  
if (income < 114650) rate = 0.25;  
if (income < 47450) rate = 0.22;
```

## Σύνοψη: Αλγοριθμική Δομή Επιλογής

- Η δομή της επιλογής επιτρέπει την ανταπόκριση σε διαφορετικές καταστάσεις με ευέλικτο τρόπο
- Κανονικά μία συγκεκριμένη εντολή επιλογής πρέπει να περιλαμβάνει δράσεις για κάθε πιθανό ενδεχόμενο που μπορεί να προκύψει, έστω και αν κάποια από αυτές τις δράσεις είναι η «απραξία» (“do nothing”) όπως συχνά συμβαίνει

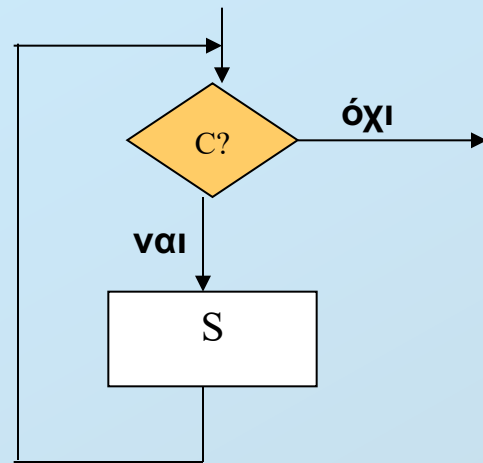
# Αλγοριθμική Δομή Επανάληψης

## Μορφές Επανάληψης

- Απροσδιόριστη Επανάληψη ή Επανάληψη υπό συνθήκη
  - εντολές **while** και **do-while**
- Απαριθμητή Επανάληψη
  - εντολή **for**

## Εντολή while

**while** (συνθήκη) εντολή



## Σημασιολογία

Εάν από την αρχή δεν ευσταθεί η συνθήκη C, η εντολή S δεν θα εκτελεστεί καθόλου. Διαφορετικά η S εκτελείται και αυτό επαναλαμβάνεται μέχρις ότου η συνθήκη C να μην ευσταθεί.

Επομένως για να μην δημιουργηθεί βρόχος άπειρης διάρκειας (infinite loop), η διεργασία της S πρέπει να είναι τέτοια, ούτως ώστε να επέλθει μία κατάσταση στην οποία η C να μην ευσταθεί πλέον, οδηγώντας έτσι σε έξοδο από το βρόχο.

**Παράδειγμα: Υπολογισμός του αθροίσματος  $1 + 2 + \dots + n$**

```
public class ComputeSum{

    public static void main(String[] args){
        int n = Integer.parseInt(args[0]);
        int count = 1, sum = 0;
        while (count <= n){
            sum = sum + count;
            count = count + 1;
        }

        System.out.println("The sum is " + sum);
    }
}
```

Παράδειγμα: Πρόγραμμα το οποίο να παράγει τα ακόλουθα:

|   |    |
|---|----|
| 0 | 1  |
| 1 | 2  |
| 2 | 4  |
| 3 | 8  |
| 4 | 16 |
| 5 | 32 |
| 6 | 64 |

|   |                |
|---|----------------|
| x | 2 <sup>x</sup> |
|---|----------------|

```
public class PowersOfTwo {  
    public static void main (String[] args){  
        int count = 0, res = 1;  
        while (count <= 6) {  
            System.out.println(count + " " + res);  
            count = count + 1;  
            res = res * 2;  
        }  
    }  
}
```

**Υπάρχει κάτι λάθος στον πιο κάτω κώδικα;**

```
public class PowersOfTwo {  
    public static void main (String[] args){  
        int count = 0, res = 1;  
        while (count <= 6)  
            System.out.println(count + " " + res);  
            count = count + 1;  
            res = res * 2;  
    }  
}
```

```
/* infinite loop */  
  
while (true) System.out.println("I cannot stop!");  
  
/* null statement */  
  
while (false) System.out.println("????");  
while (false);
```

## Σύνθετοι Τελεστές Ανάθεσης

```
count = count + 1;  
res = res * 2;  
sum = sum + count;  
x = x - 3;  
n = n * (x+1);
```

μεταβλητή **τελεστής** = έκφραση;

Σημασιολογία:

μεταβλητή = μεταβλητή τελεστής (έκφραση);

Σύνθετοι τελεστές: +=, -=, \*=, /=, %=



```
count += 1;  
res *= 2;  
sum += count;  
x -= 3;  
n *= (x+1);
```

```
int count = 1, sum = 0;
while (count <= n) {
    sum += count;
    count += 1;
}
```

```
int count = 0, res = 1;
while (count <= 6) {
    System.out.println(count + " " + res);
    count += 1;
    res *= 2;
}
```

## Εντολή for

**for** (έκφραση-αρχικοποίησης ;  
συνθήκη ;  
έκφραση-ενημέρωσης)  
εντολή

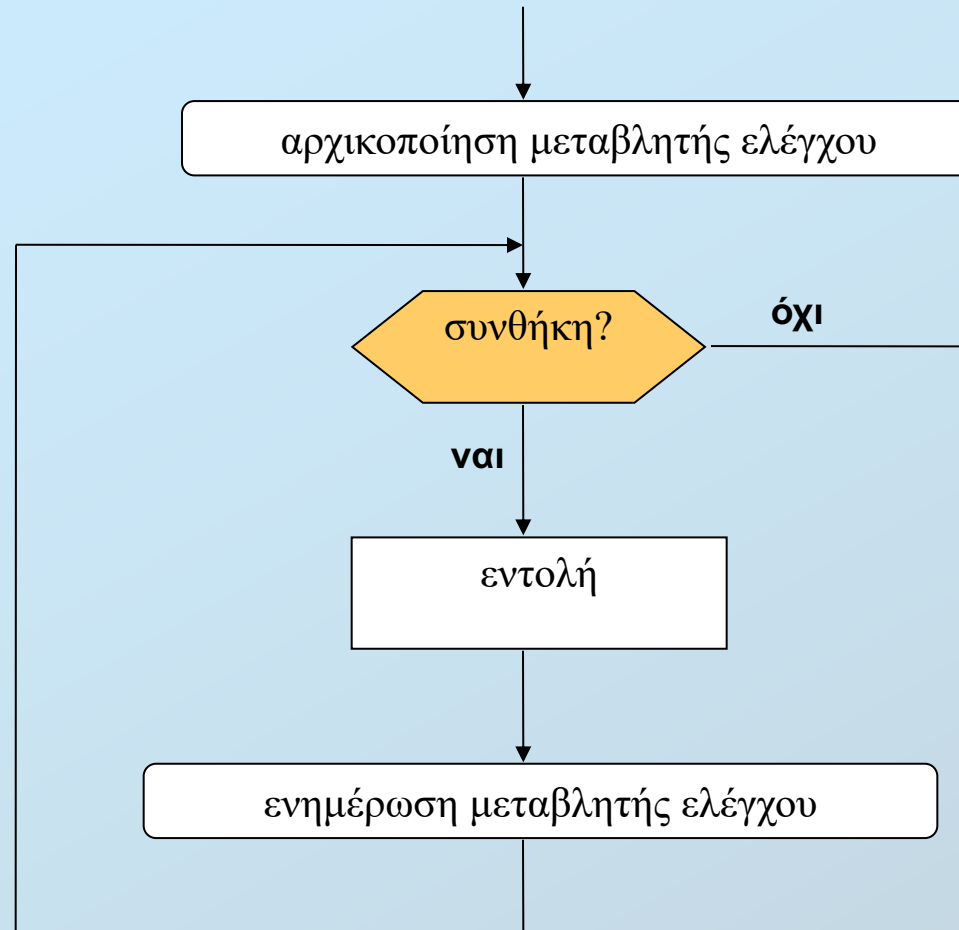
### Γενική Σημασιολογία

Καταρχήν αποτιμάται η **έκφραση-αρχικοποίησης**, και στη συνέχεια δοκιμάζεται η **συνθήκη**. Εάν δεν ευσταθεί, ο έλεγχος μεταφέρεται εκτός της for.

Διαφορετικά εκτελείται η **εντολή**, αποτιμάται η **έκφραση-ενημέρωσης** και ο έλεγχος επιστρέφει στη δοκιμή της συνθήκης.

Η εκτέλεση της εντολής και η αποτίμηση της έκφρασης-ενημέρωσης επαναλαμβάνεται ενόσω η συνθήκη ευσταθεί. Διαφορετικά τερματίζεται ο βρόγχος.

## Τυπική Εφαρμογή της Εντολής for



```
/* Examples of infinite loops */  
for (int n=1; true; n += 1)  
    System.out.println("I cannot stop!");  
  
for (; true; ) System.out.println("I cannot stop!");
```

```
/* Examples of null statements */  
for (int n = 1; n == 10; n += 1)  
    System.out.println("???");  
  
for (; false; ) System.out.println("???");  
  
-- Unreachable code - compile error
```

**Παράδειγμα:** Υπολογισμός του αθροίσματος  $1 + 2 + \dots + n$

```
public class ComputeSum{  
  
    public static void main(String[] args){  
        int n = Integer.parseInt(args[0]);  
        int sum = 0;  
        for (int count = 1; count <= n; count += 1)  
            sum = sum + count;  
  
        System.out.println("The sum is " + sum);  
    }  
}
```

**Παράδειγμα:** Πρόγραμμα το οποίο εκτυπώνει  $2^x$  για  $x$  από το 1 μέχρι το 6

```
public class PowersOfTwo {  
    public static void main (String[] args){  
        int res = 1;  
        for (int count = 0; count <= 6; count += 1) {  
            System.out.println(count + " " + res);  
            res *= 2;  
        }  
    }  
}
```

## Τελεστές για Αύξηση και Μείωση Τιμών (++ , —)

### Increment and Decrement Operators

```
count = count + 1;  
count += 1;  
++count
```

Ο **τελεστής αύξησης**, ++, λαμβάνει μία μοναδική μεταβλητή ως τον τελεστέο του

Το πλευρικό φαινόμενο της εφαρμογής του, είναι ότι η τιμή του τελεστέου του αυξάνεται κατά ένα

Συνήθως ο τελεστής χρησιμοποιείται απλά για αυτή την παρενέργεια, όπως στο παρακάτω παράδειγμα:

```
for (int count = 1; count <= n; ++count) sum += count;
```

## Παρατηρήσεις

Η τιμή μίας έκφρασης στην οποία χρησιμοποιείται ο τελεστής ++, εξαρτάται από τη θέση του τελεστή:

### Προ-σημειογραφική αύξηση (prefix increment)

ο τελεστής προηγείται του τελεστέου, π.χ. ++count  
η τιμή της έκφρασης είναι η τιμή του τελεστέου **μετά** την αύξηση

### Μετα-σημειογραφική αύξηση (postfix increment)

ο τελεστής ακολουθεί τον τελεστέο, π.χ. count++  
η τιμή της έκφρασης είναι η τιμή του τελεστέου **πριν** την αύξηση

Παρομοίως, ο **τελεστής μείωσης**, --, λαμβάνει μία μοναδική μεταβλητή ως τελεστέο, η παρενέργεια της εφαρμογής του είναι η μείωση της τιμής του τελεστέου κατά ένα, ενώ παράλληλα επιστρέφεται η τιμή του τελεστέου μετά ή πριν τη μείωση, σύμφωνα με το εάν χρησιμοποιείται προ- ή μετα-σημειογραφική μείωση αντιστοίχως

## Παράδειγμα



`j = ++i;`

Προ-σημειογραφική  
αύξηση:

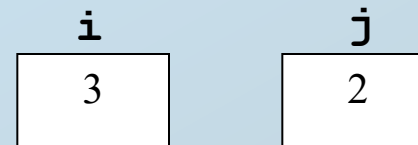
Πρώτα αυξάνεται η `i` και  
μετά χρησιμοποιείται η  
τιμή της



`j = i++;`

Μετά-σημειογραφική  
αύξηση:

Πρώτα χρησιμοποιείται η  
τιμή της `i` και μετά γίνεται  
η αύξηση



## Επιστροφή στο Παράδειγμα των Υποδιαιρέσεων μιας Ρήγας

```
public class Ruler {  
    public static void main(String[] args) {  
        int N = Integer.parseInt(args[0]);  
        String ruler = " ";  
        for (int i = 1; i <= N; i++) {  
            ruler = ruler + i + ruler;  
        }  
        System.out.println(ruler);  
    }  
}
```

| i | ruler             |
|---|-------------------|
|   | " "               |
| 1 | " 1 "             |
| 2 | " 1 2 1 "         |
| 3 | " 1 2 1 3 1 2 1 " |

```
% java Ruler 1
```

```
1
```

```
% java Ruler 2
```

```
1 2 1
```

```
% java Ruler 3
```

```
1 2 1 3 1 2 1
```

```
% java Ruler 4
```

```
1 2 1 3 1 2 1 4 1 2 1 3 1 2 1
```

```
% java Ruler 5
```

```
1 2 1 3 1 2 1 4 1 2 1 3 1 2 1 5 1 2 1 3 1 2 1 4 1 2 1 3 1 2 1
```

```
% java Ruler 100
```

```
Exception in thread "main"
```

```
java.lang.OutOfMemoryError
```

## Παράδειγμα: Παρουσίαση των εκτυπώσιμων χαρακτήρων με τους κωδικούς τους

```
public class PrintChars {  
  
    public static void newLine(){  
        System.out.println();  
    }  
  
    public static void main(String[] args){  
        char START_CHAR = ' ';  
        char END_CHAR = '~';  
        int per_line = 5;  
        newLine();  
        System.out.println("Displaying Print Characters with their Codes");  
        for (char c = START_CHAR; c <= END_CHAR; c++){  
            if ((c - START_CHAR) % per_line == 0) newLine();  
            System.out.print(" " + c + " " + (int)c);  
        }  
    }  
}
```

```
$ java PrintChars
```

### Displaying Print Characters with their Codes

```
 32 ! 33 " 34 # 35 $ 36  
% 37 & 38 ' 39 ( 40 ) 41  
* 42 + 43 , 44 - 45 . 46  
/ 47 0 48 1 49 2 50 3 51  
4 52 5 53 6 54 7 55 8 56  
9 57 : 58 ; 59 < 60 = 61  
> 62 ? 63 @ 64 A 65 B 66  
C 67 D 68 E 69 F 70 G 71  
H 72 I 73 J 74 K 75 L 76  
M 77 N 78 O 79 P 80 Q 81  
R 82 S 83 T 84 U 85 V 86  
W 87 X 88 Y 89 Z 90 [ 91  
\ 92 ] 93 ^ 94 _ 95 ` 96  
a 97 b 98 c 99 d 100 e 101  
f 102 g 103 h 104 i 105 j 106  
k 107 l 108 m 109 n 110 o 111  
p 112 q 113 r 114 s 115 t 116  
u 117 v 118 w 119 x 120 y 121  
z 122 { 123 | 124 } 125 ~ 126
```

### Displaying Print Characters with their Codes

|   |     |   |     |   |     |   |     |    |     |
|---|-----|---|-----|---|-----|---|-----|----|-----|
|   | 32  | ! | 33  | " | 34  | # | 35  | \$ | 36  |
| % | 37  | & | 38  | ' | 39  | ( | 40  | )  | 41  |
| * | 42  | + | 43  | , | 44  | - | 45  | .  | 46  |
| / | 47  | 0 | 48  | 1 | 49  | 2 | 50  | 3  | 51  |
| 4 | 52  | 5 | 53  | 6 | 54  | 7 | 55  | 8  | 56  |
| 9 | 57  | : | 58  | ; | 59  | < | 60  | =  | 61  |
| > | 62  | ? | 63  | @ | 64  | A | 65  | B  | 66  |
| C | 67  | D | 68  | E | 69  | F | 70  | G  | 71  |
| H | 72  | I | 73  | J | 74  | K | 75  | L  | 76  |
| M | 77  | N | 78  | O | 79  | P | 80  | Q  | 81  |
| R | 82  | S | 83  | T | 84  | U | 85  | V  | 86  |
| W | 87  | X | 88  | Y | 89  | Z | 90  | [  | 91  |
| \ | 92  | ] | 93  | ^ | 94  | _ | 95  | `  | 96  |
| a | 97  | b | 98  | c | 99  | d | 100 | e  | 101 |
| f | 102 | g | 103 | h | 104 | i | 105 | j  | 106 |
| k | 107 | l | 108 | m | 109 | n | 110 | o  | 111 |
| p | 112 | q | 113 | r | 114 | s | 115 | t  | 116 |
| u | 117 | v | 118 | w | 119 | x | 120 | y  | 121 |
| z | 122 | { | 123 |   | 124 | } | 125 | ~  | 126 |

```
System.out.print(" " + c + " " + (int)c);
```

```
System.out.printf("\t%c%4d", c, (int)c);
```

```

public class PrintChars {

    public static void newLine () {
        System.out.println();
    }

    // Enter characters from command line
    public static void main(String[] args) {
        String s1 = args[0];
        String s2 = args[1];
        char START_CHAR = s1.charAt(0); // args[0].charAt(0);
        char END_CHAR = s2.charAt(0);   // args[1].charAt(0);
        int per_line = 5;
        newLine();
        System.out.println("Displaying Print Characters with their Codes");
        for (char c = START_CHAR; c <= END_CHAR; c++) {
            if ((c - START_CHAR) % per_line == 0) newLine();
            System.out.print(" " + c + " " + (int)c);
        }
    }
}

```

```
$ java PrintChars a A
```

```
Displaying Print Characters with their Codes
```

```

public class PrintChars {

    public static void newLine () {
        System.out.println();
    }

    // Enter characters from command line
    public static void main(String[] args) {
        String s1 = args[0];
        String s2 = args[1];
        char START_CHAR = s1.charAt(0); // args[0].charAt(0);
        char END_CHAR = s2.charAt(0);   // args[1].charAt(0);
        if (START_CHAR > END_CHAR)
            {char temp = START_CHAR; START_CHAR = END_CHAR; END_CHAR = temp;}
        int per_line = 5;
        newLine();
        System.out.println("Displaying Print Characters with their Codes");
        for (char c = START_CHAR; c <= END_CHAR; c++)
            if ((c - START_CHAR) % 5 == 0)
                System.out.println();
        }
    }
}

```

```
$ java PrintChars a A
```

```
Displaying Print Characters with their Codes
```

```

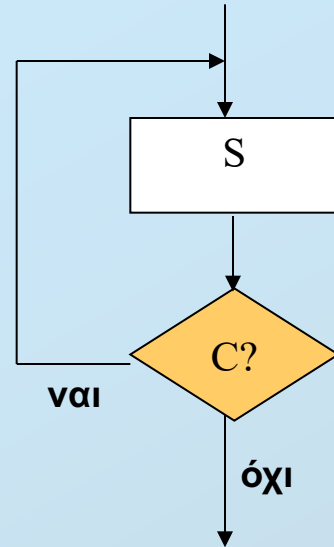
A 65 B 66 C 67 D 68 E 69
F 70 G 71 H 72 I 73 J 74
K 75 L 76 M 77 N 78 O 79
P 80 Q 81 R 82 S 83 T 84
U 85 V 86 W 87 X 88 Y 89
Z 90 [ 91 \ 92 ] 93 ^ 94
_ 95 ` 96 a 97

```

## Εντολή do-while

do εντολή while (συνθήκη) ;

do S while (C);



### Σημασιολογία

Καταρχήν εκτελείται η εντολή S και μετά δοκιμάζεται η συνθήκη C

Ενόσω αυτή επαληθεύεται η εκτέλεση της εντολής S επαναλαμβάνεται

Επομένως η S εκτελείται τουλάχιστον μια φορά

Προς αποφυγή βρόγχου άπειρης διάρκειας, η διεργασία της S πρέπει να είναι τέτοια ούτως ώστε να επέλθει κάποια κατάσταση στην οποία η C να μην υφίσταται πλέον

**Έλεγχος εγκυρότητας δεδομένων:** Η εντολή do-while είναι συνήθως η πιο κατάλληλη δομή για τον έλεγχο εγκυρότητας δεδομένων

```
Scanner in= new Scanner(System.in);  
int A, B;  
do {  
    System.out.print("Enter values for A and B where A < B");  
    A = in.nextInt();  
    B = in.nextInt();  
} while (A >= B);
```

**Παράδειγμα:** Εισδοχή 10 αριθμών από το πεδίο [0.0, 10.0] και υπολογισμός του μέσου όρου αυτών των αριθμών

```
double sum = 0.0, num;
for (int i=1; i <= 10; ++i){
    do
        num = StdIn.readDouble();
    while (num < 0.0 || num > 10.0);
    sum += num;
}
StdOut.println("The average is" + sum/10);
```

**Παράδειγμα:** Εισδοχή 10 αριθμών από το πεδίο [0.0, 10.0] και υπολογισμός του μέσου όρου αυτών των αριθμών

### Εναλλακτικός Ορισμός

```
double sum = 0.0, num;
int count = 0;
while (count < 10){
    num = StdIn.readDouble();
    if (num >= 0.0 && num <= 10.0){
        count++; sum += num;
    }
}
StdOut.println("The average is" + sum/10);
```

```
do
    System.out.println("I cannot stop!");
while (true);
-- Unreachable code - compile error
```

```
do
    System.out.println("I will stop immediately!");
while (false);
```

```
≡ System.out.println("I will stop immediately!");
```

# Ισοδυναμίες

**while, do, for**

```
while ( $\sigma$ ) {  $\beta$  }
```

```
if ( $\sigma$ )  
  do {  $\beta$  } while ( $\sigma$ );
```

```
for ( ;  $\sigma$ ; ) {  $\beta$  }
```

```
do { $\beta$ } while ( $\sigma$ );
```

```
{  $\beta$  }  
while ( $\sigma$ ) {  $\beta$  }
```

```
{  $\beta$  }  
for ( ;  $\sigma$ ; ) {  $\beta$  }
```

```
for ( $\alpha$ ;  $\sigma$ ;  $\varepsilon$ ) {  $\beta$  }
```

```
 $\alpha$ ;  
if ( $\sigma$ )  
do {  $\beta$ ;  $\varepsilon$  } while ( $\sigma$ );
```

```
 $\alpha$ ;  
while ( $\sigma$ ) {  $\beta$ ;  $\varepsilon$  }
```

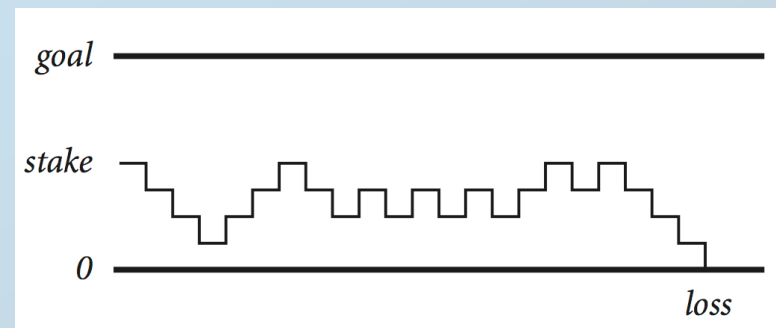
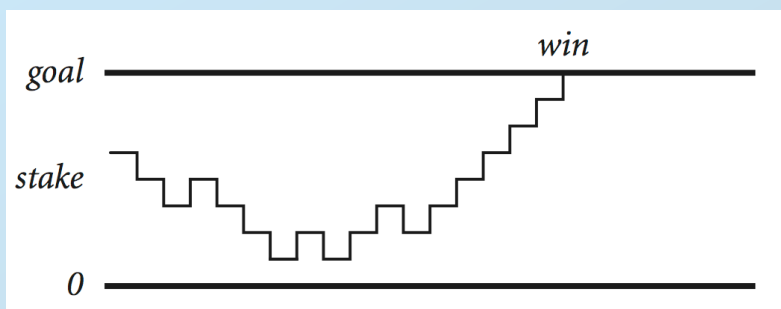
# Προσομοίωση Monte Carlo

---



# Παράδειγμα: Η Καταστροφή του Τζογαδόρου

- Ο τζογαδόρος αρχίζει με ένα ποσό  $\text{€stake}$  και βάζει στοιχήματα του  $\text{€1}$  μέχρις ότου είτε να εξαντλήσει το ποσό των  $\text{€stake}$  ή να φτάσει το στόχο του των  $\text{€goal}$ .
  - Τι πιθανότητες έχει να κερδίσει;
  - Πόσα στοιχήματα πρέπει να παίξει;
- Μια προσέγγιση: Προσομοίωση Monte Carlo
  - Στρίψτε ψηφιακά κέρματα και παρατηρείστε τι συμβαίνει
  - Επαναλάβετε και υπολογίστε τις στατιστικές



# Η Καταστροφή του Τζογαδόρου

```
public class Gambler {  
    public static void main(String[] args) {  
        int stake = Integer.parseInt(args[0]);  
        int goal = Integer.parseInt(args[1]);  
        int trials = Integer.parseInt(args[2]);  
        int wins = 0;  
        // repeat experiment N times  
        for (int i = 0; i < trials; i++) {  
            // do one gambler's ruin experiment  
            int t = stake;  
            while (t > 0 && t < goal) {  
                // flip coin and update  
                if (Math.random() < 0.5) t++;  
                else t--;  
            }  
            if (t == goal) wins++;  
        }  
        System.out.println(wins + " wins of " + trials);  
    }  
}
```

stake goal trials



```
% java Gambler 5 25 1000
```

```
191 wins of 1000
```

```
% java Gambler 5 25 1000
```

```
203 wins of 1000
```

```
% java Gambler 500 2500 1000
```

```
197 wins of 1000
```

Μετά από αρκετή ώρα....

**Γεγονός:** Πιθανότητα νίκης =  $\text{stake} \div \text{goal}$ .

**Γεγονός:** Αναμενόμενος αριθμός στοιχημάτων =  $\text{stake} \times \text{desired gain}$ .

**Παράδειγμα:** 20% πιθανότητα να μετατρέψεις τα €500 σε €2500,  $500/2500 = 20\%$   
αλλά να αναμένεις να κάνεις ένα εκατομμύριο στοιχήματα του €1.  $500 * (2500 - 500) = 1 \text{ εκατομμύριο}$

Γινόμενο του stake με το  
επιθυμητό κέρδος

```

public class Gambler {

    private static boolean Win(int s, int g){
        while (s > 0 && s < g){
            if (Math.random() < 0.5) s++;
            else s--;
        }
        return s == g;
    }

    public static void main(String[] args) {
        int stake  = Integer.parseInt(args[0]);
        int goal    = Integer.parseInt(args[1]);
        int trials  = Integer.parseInt(args[2]);
        int wins    = 0;

        for (int i = 0; i < trials; i++) {
            if (Win(stake, goal)) wins++;
        }

        System.out.println(wins + " wins of " + trials);
    }
}

```

# Κοινά Λάθη

- Σύγχυση ανάμεσα στις εντολές if και while:
  - if (συνθήκη) εντολή
  - while (συνθήκη) εντολή
- Παράλειψη των παρενθέσεων γύρω από τις συνθήκες, στις εντολές while και do-while
- Σχετικά με την εντολή for, παράλειψη του χαρακτήρα ; μετά την έκφραση αρχικοποίησης και μετά την συνθήκη επανάληψης
- Σε κάθε περίπτωση, το σώμα της επανάληψης θεωρείται ότι είναι μία και μοναδική εντολή
  - Κοινό λάθος είναι η παράλειψη των παρενθέσεων { } όταν το τι επαναλαμβάνεται είναι μία ακολουθία εντολών όπως φαίνεται στην ακόλουθη διαφάνεια

```
while (x > x_big)
    x -= z;
    ++x_big;
```

Σε αυτή την εντολή το σώμα της επανάληψης θεωρείται μόνο η εντολή `x -= z;`

```
while (x > x_big)
{
    x -= z;
    ++x_big;}
```

Εδώ, η ακολουθία των δύο εντολών αποτελεί το σώμα της επανάληψης

# Κοινά Λάθη

- **if (συνθήκη<sub>1</sub>)**  
    **do {**  
        ..... **}**  
    **while (συνθήκη<sub>1</sub>);**

θα πρέπει να απλοποιηθεί σε

**while (συνθήκη<sub>1</sub>)**  
    **{ ..... }**

# Κοινά Λάθη

- Η χρήση του τελεστή για μη ισότητα αναφορικά με μεταβλητές τύπου double πρέπει να αποφεύγεται, π.χ.

```
while (balance != 0.0)
```

```
    . . . . .
```

Αυτή η συνθήκη πολύ πιθανώς ενδέχεται να οδηγήσει σε βρόγχο άπειρης διάρκειας, απλά επειδή η τιμή 0.0 μπορεί να μην καταχωρηθεί επακριβώς στη μεταβλητή balance

```
while (balance > 0.0)
```

```
    . . . . .
```

Εδώ αποφεύγεται ο βρόγχος, νοουμένου ότι η τιμή της balance δεν παραμένει συνεχώς κάποιος θετικός αριθμός

# Κοινά Λάθη

- Παρερμηνευση των σύνθετων τελεστών ανάθεσης

$M \tau = E$       σημαίνει       $M = M \tau (E)$   
                         και όχι               $M = M \tau E$

Συνεπώς, οι παρενθέσεις γύρω από την έκφραση  $E$  που αποτελεί τον δεξιό τελεστέο αποτελούν αναγκαίο στοιχείο

π.χ.               $a *= b + c;$   
σημαίνει       $a = a * (b + c);$   
και όχι               $a = a * b + c; \equiv a = (a * b) + c;$

Δεν υπάρχει συντομογραφία για αυτή την πρόταση

$a *= b;$                $a += c;$

# Σύνοψη Παραδειγμάτων Επανάληψης

|                                                                                          |                                                                                                                          |
|------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <i>print powers of two</i>                                                               | <pre>int v = 1; for (int i = 0; i &lt;= N; i++) {     System.out.println(i + " " + v);     v = 2*v; }</pre>              |
| <i>print largest power of two less than or equal to N</i>                                | <pre>int v = 1; while (v &lt;= N/2)     v = 2*v; System.out.println(v);</pre>                                            |
| <i>compute a finite sum<br/>(<math>1 + 2 + \dots + N</math>)</i>                         | <pre>int sum = 0; for (int i = 1; i &lt;= N; i++)     sum += i; System.out.println(sum);</pre>                           |
| <i>compute a finite product<br/>(<math>N! = 1 \times 2 \times \dots \times N</math>)</i> | <pre>int product = 1; for (int i = 1; i &lt;= N; i++)     product *= i; System.out.println(product);</pre>               |
| <i>print a table of function values</i>                                                  | <pre>for (int i = 0; i &lt;= N; i++)     System.out.println(i + " " + 2*Math.PI*i/N);</pre>                              |
| <i>print the ruler function<br/>(see Program 1.2.1)</i>                                  | <pre>String ruler = " "; for (int i = 1; i &lt;= N; i++)     ruler = ruler + i + ruler; System.out.println(ruler);</pre> |

# Σύνοψη

- Έλεγχος ροής:
  - Ακολουθία προτάσεων/εντολών που εκτελούνται σε ένα πρόγραμμα
  - Συνθήκες και βρόγχοι: ρυθμίζουν τον έλεγχο ροής
- Επιλογή
  - `if`, `if-else`, `switch`
- Επανάληψη
  - `while`, `for`, `do-while`

# Στο βιβλίο

- **Ενότητα 1.3**